

Universitatea Tehnica din Cluj-Napoca
Departament Calculatoare

Programming Techniques in Java

Design Patterns

T. Cioara, V. Chifu, C. Pop, D. Mitrea
2026

Design Patterns

Recurring solution to a design problem you see over and over

Classification according to GoF*

Creational

Help in object creation using indirect creation levels (subclasses or other objects)

Structural

Help in composing groups of objects into larger structures

Behavioral

Help in defining the communication between objects and also help in flow control of complex programs

* GoF - Gamma, Helm, Johnson, Vlissides (1995)

- discovered, classified and catalogued a set of 23 patterns
- Design Patterns. Elements of Reusable OO Software, Addison Wesley, 1995

Pattern Description (GoF Proposal)

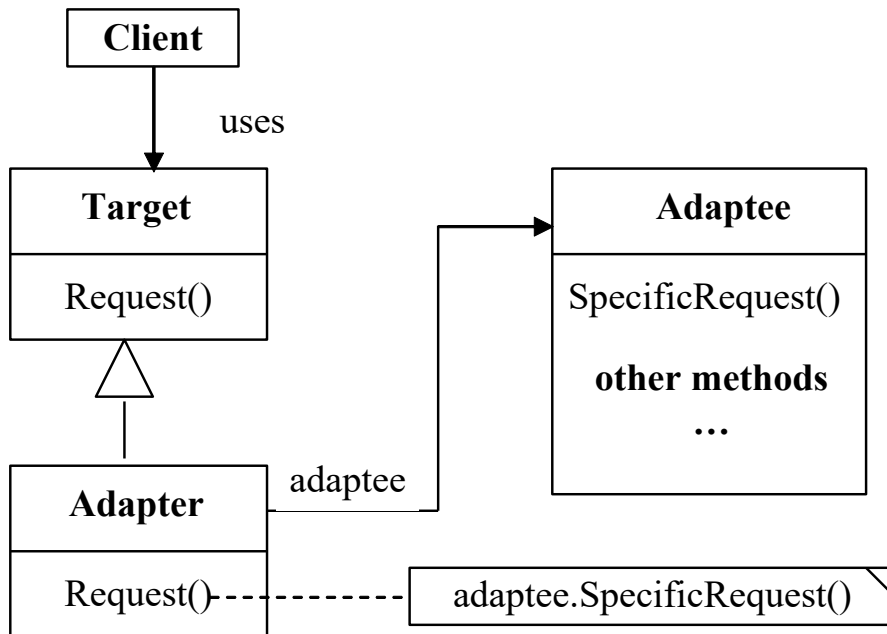
		Element	Description
Essential		Name	A good pattern name is essential
		Problem	Describes typical pattern application problems
		Solution	- Shows how the problem could be solved - Describes the design pattern elements, their relationships and responsibilities
		Consequences	- When & how the pattern should be used - Best cases scenarios to use the pattern - Benefits and cost
		Intention	A short explanation of why the pattern is useful
Extra		Applicability	Pattern application constraints
		Examples	Real problems where the pattern can be used
		Sample code	A real implementation of the pattern
		Related Patterns	

Adapter

Intention

- Convert the interface of a class into another interface that the clients expect
- Allow classes work together that couldn't otherwise, due to incompatible interface

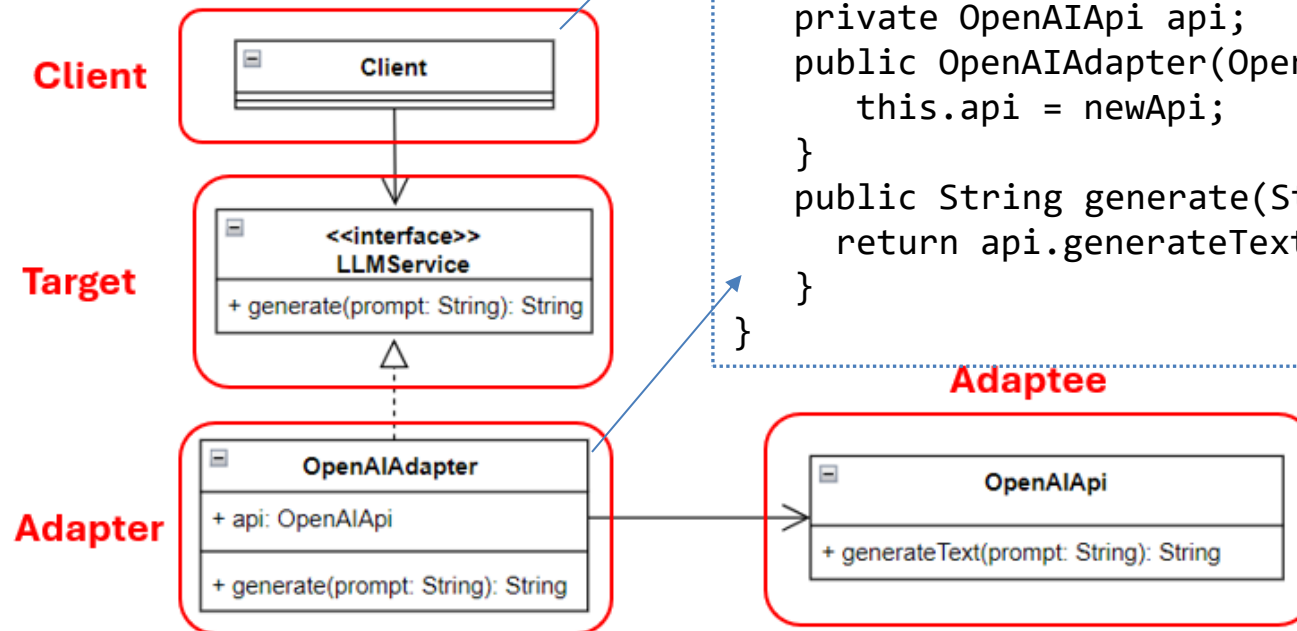
Structure



Participant	Role
Client	Class that calls a method of another class through an interface
Target	Defines the domain-specific interface that Client uses (calls)
Adapter	• Adapts the interface of Adaptee to the Target interface • Its methods call one or more methods that class Adaptee defines
Adaptee	• This class doesn't implement the Target methods but has some functionalities you want the Client class to call or, • Defines an existing interface that needs adapting

Adapter

- Example – Application using LLM services
 - Integrating an LLM Service that does not implement the interface the Application expects



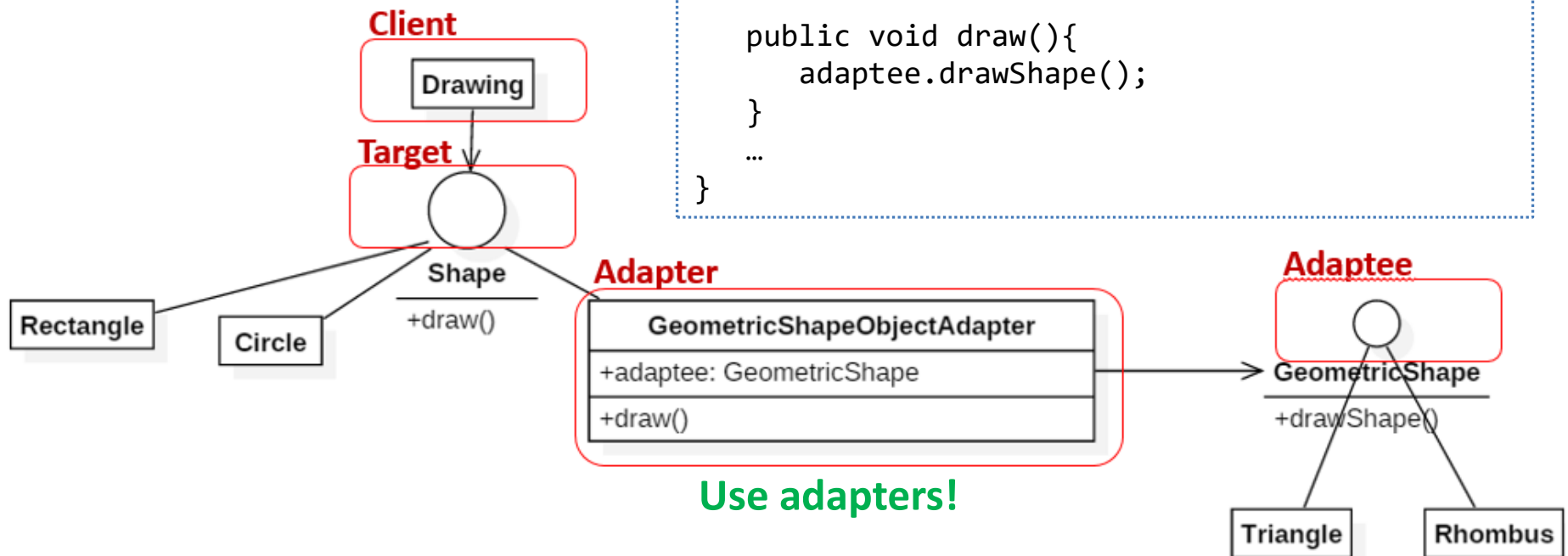
```
LLMService llm = new OpenAIAdapter(new  
OpenAIApi()); llm.generate("Hello");
```

```
class OpenAIAdapter implements LLMService {  
    private OpenAIApi api;  
    public OpenAIAdapter(OpenAIApi newApi){  
        this.api = newApi;  
    }  
    public String generate(String prompt) {  
        return api.generateText(prompt);  
    }  
}
```

Use Adapters!

Adapter

- Example – Drawing and arranging graphical elements into diagrams ([Source](#))
 - Integrating a 3rd party component



Adapter

- **Consequences**
 - *Adapter* working with many *Adaptees*
 - Eventually with the subclasses of a class *Adaptee*
 - The *Adapter* can add functionality to the *Adaptee* classes
 - What is the adaptation level?
 - Depends on how similar is the Target interface to *Adaptee's* interface
 - Simple interface conversion
 - operation name conversion
 - Different set of operations

Adapter

Adapters in Java event handling

```
public class MyClass implements MouseListener {  
    ... someObject.addMouseListener(this);  
    ...  
    public void mouseReleased(MouseEvent e) { // empty def.}  
    public void mouseClicked(MouseEvent e) {  
        //listener implementation goes here... }  
}
```

Using an adapter class

```
public class MyClass extends JFrame{  
    ....  
    MyHandlerClass mh= new MyHandlerClass();  
    addMouseListener(mh);  
}  
  
public class MyHandlerClass extends MouseAdapter {  
    ....  
    public void mouseClicked(MouseEvent e) {  
        //Event listener implementation goes here}  
}
```

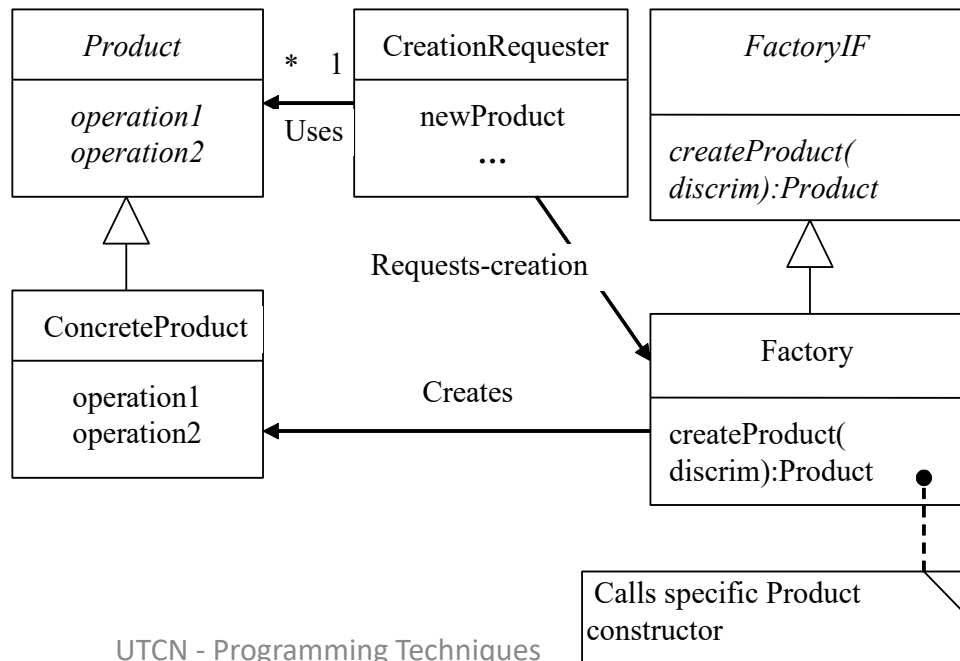
***public abstract class MyHandlerClass extends Object implements
MouseListener, MouseMotionListener {...}***

Factory Method

Intention

- A class should instantiate other classes without being dependent on them
 - Delegates the choice of which class to instantiate to another object
 - The newly created object is referred through a common interface

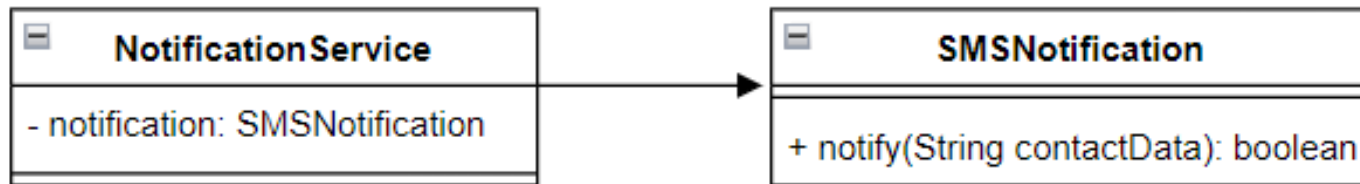
Structure



Participant	Role
Product	The abstract super-class of objects produced by the Factory Method pattern
Concrete Product	Concrete class instantiated by the objects that participate in the Factory Method pattern
FactoryIF	Application independent abstract class – its sub-classes create <i>Product</i> objects on behalf of <i>CreationRequester</i>
Factory	Subclass of <i>FactoryIF</i> that has a method to create <i>ConcreteProduct</i> objects.
Creation Requester	Needs to create application specific objects. It does so indirectly through an instance of a factory class.

Factory Method

Example – Service for sending notifications to clients

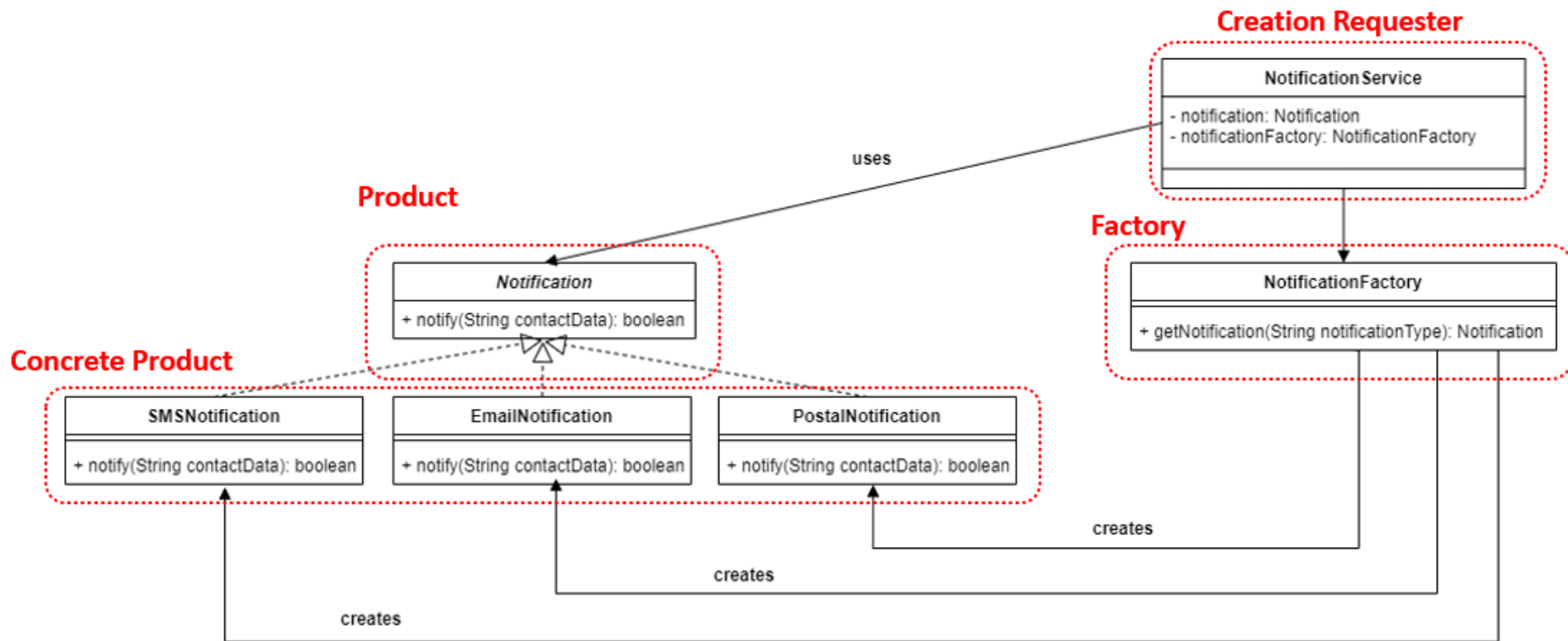


- Currently the service supports only SMS notification
- Want to include other types of notifications (e.g., e-mail, postal, etc.)
 - how should the *NotificationService* change so that it does not depend on concrete notifications?

Adapted from [Source](#)

Factory Method

Example – Service for sending notifications to clients



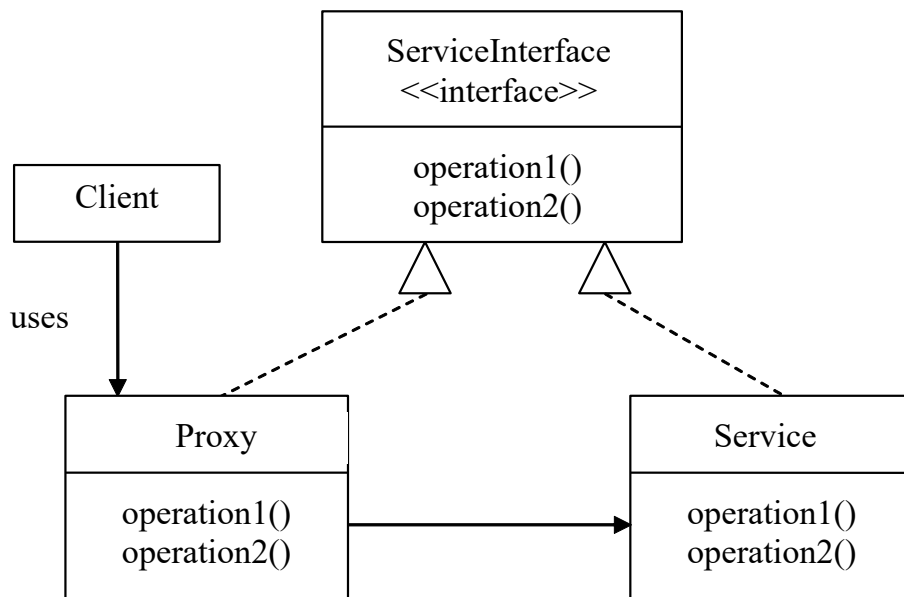
Adapted from [Source](#)

Proxy

Intention

- Creation of surrogate objects that replace other objects
 - Usually expensive or that need not an immediate instantiation
- Proxy clients are not aware they are dealing with surrogate objects

Structure



Participant	Role
Service	Provides the full service.
Client	• Uses the services provided by class Service. • It cannot directly access Service operations and resources, but by means of Proxy class.
Proxy	An indirect resource between client and service. Postpones the instantiation of Service objects until they are needed (Virtual Proxy).
Service Interface	Interface implemented by Proxy and Service.

Proxy

Proxy Types

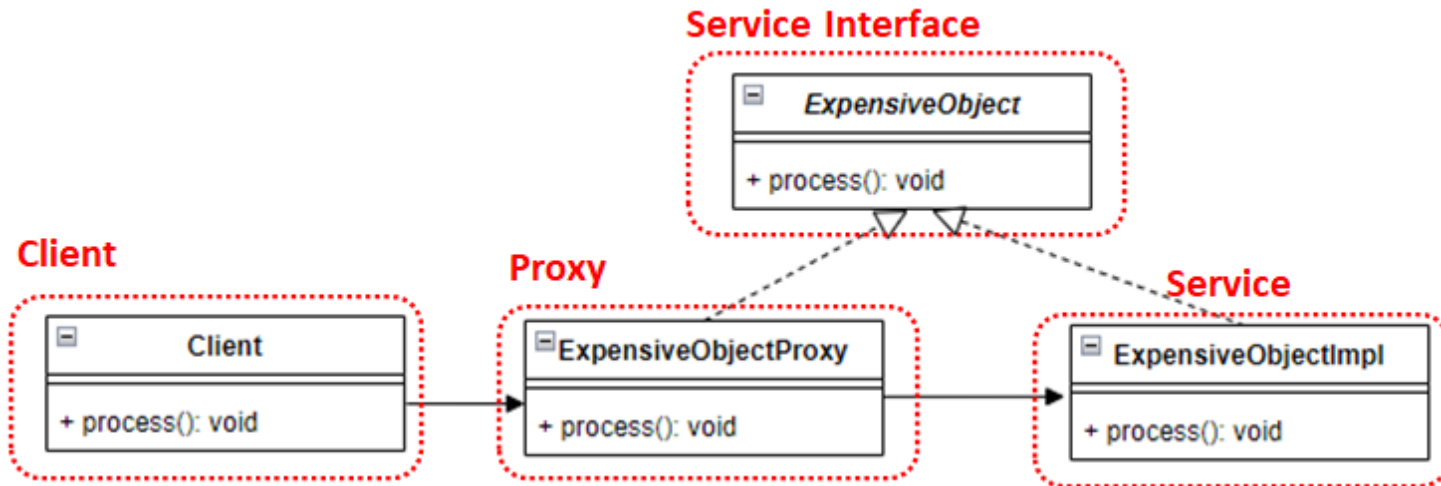
Type	Description
Representative of an expensive operation	The operation takes a long execution time. The Proxy returns a quick partial answer.
Remote proxy	Representative of an operation implemented on a remote machine.
Access proxy	Access control to shared resources
Virtual proxy	Representative of an expensive object. Postpones the instantiation of expensive objects until it is required (lazy instantiation).

Consequences

- The clients are not aware of Proxy arrangements
- Classes that are accessed through proxy are dynamically loaded

Proxy

- Example – Managing heavy Java objects (e.g., JDBC connection)



```
public class ExpensiveObjectProxy implements ExpensiveObject {  
    private static ExpensiveObject object;  
    @Override  
    public void process() {  
        if (object == null) {object = new ExpensiveObjectImpl();}  
        object.process();  
    }  
}
```

[Source](#)

Singleton

- **Intention**

- The creation of only one instance object of a class (the singleton class)
- The class will be responsible for the management of its sole instance
- Singleton class hides the one only instance creation operation behind a static member method
 - Traditionally named `instance()`
 - Method `instance()` returns a reference (or pointer) to the sole instance

Singleton

Implementation using static methods in Java

If the oneObject already exists, the programmer always obtains the same reference

```
public final class Singleton {  
    private static Singleton oneObject;  
    private Singleton() { }  
    public static Singleton getInstance() {  
        if(oneObject == null)  
            oneObject = new Singleton();  
        return oneObject;  
    }  
}  
...  
Singleton s1 = Singleton.getInstance();  
Singleton s2 = Singleton.getInstance();  
// s1 and s2 are the same object  
// (same reference)
```

Observer

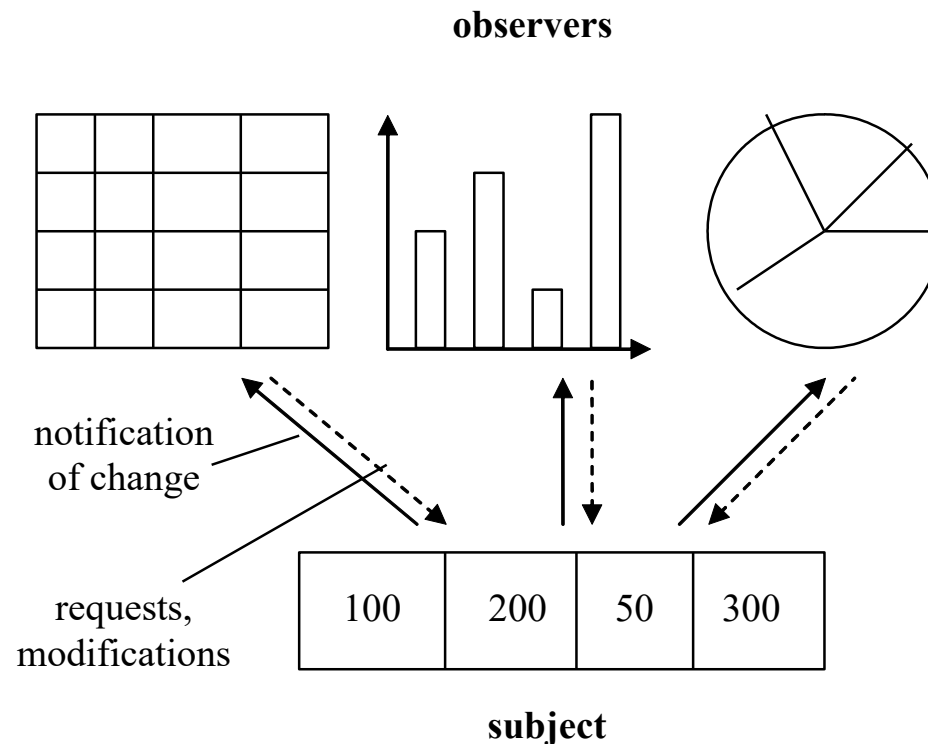
- When data changes, who needs to know?
 - Observer idea: Instead of observers constantly asking "Did something change?", the subject tells them.
- Publisher subscriber
 - What if we have 1,000 observers?
 - What if they're on different machines?
- Streaming Systems
 - What if data never stops?
- AI Pipelines / LLM Chat Systems

If you understand Observer, you understand how modern AI systems stay synchronized, scalable, and modular.

Observer

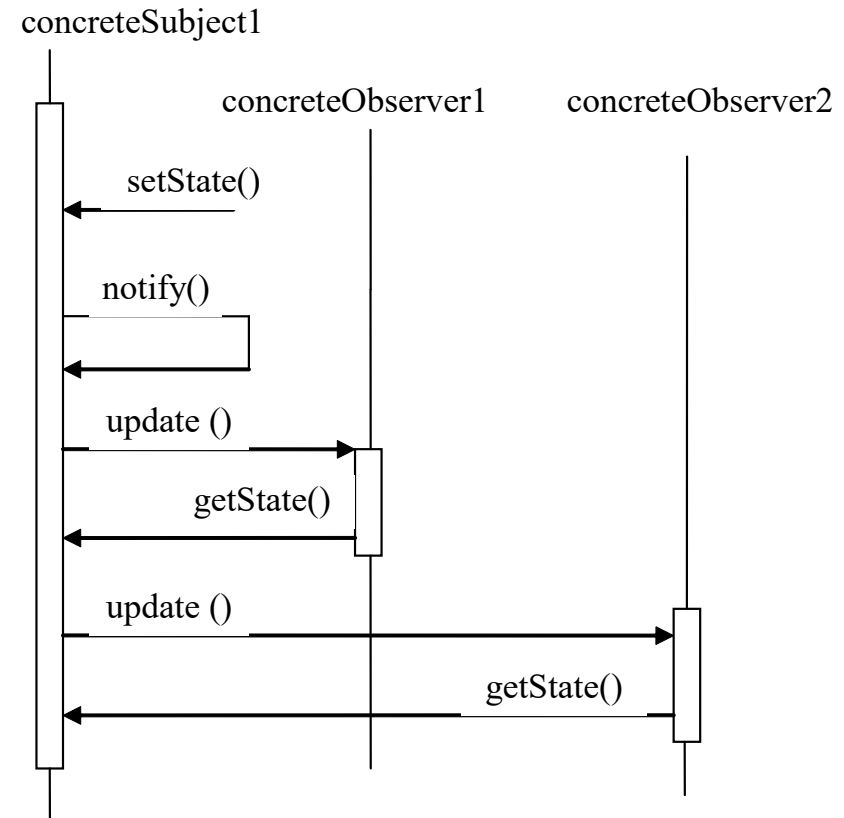
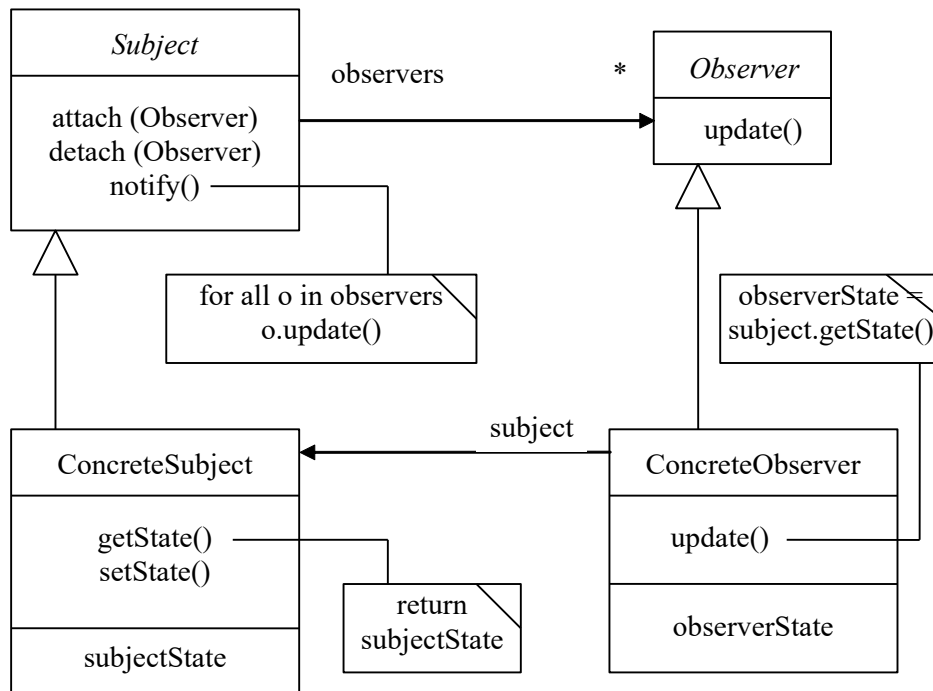
- **Intention**

- Defines 1 : n dependency between objects
- When the one side object changes the state, all its n dependent objects are automatically informed and updated



Observer

- Structure and participants



Observer Design

- **Structure and participants**

Participant	Description
Subject	• Is associated with many dependent Observers => many Observer objects may observe the Subject object
ConcreteSubject	• Stores a state in which the observers are interested • Notifies their observers whenever the state changes

Participant	Description
Observer	• Specifies an interface that should be implemented by ConcreteObservers that should be notified whenever a change occurs in the observed subjects
ConcreteObserver	• Implements the Observer's updating interface • Are notified whenever a subject changes its state • Manages a reference to a ConcreteSubject in which it shows interest • Stores a state that should be consistent with the associated ConcreteSubject

Observer

- **Use the pattern**
 - When a certain abstraction has many dependent components
 - When changes to an object determines changes of other dependable objects (the number of dependable objects is unknown)
- **Main consequences**
 - Because of weak coupling between subjects and observers
 - Observers can be added / removed without modifying their subjects
 - Subjects and Observers can belong to different abstraction layers
 - Support for event broadcasting
 - Subject sends notifications to all their subscribed observers

Observer

- **Example – Products management (inheritance-based approach)**

The Subject

```
public class Item extends Observable {
    private String name;
    private int stock;
    private double price;
    public Item (String s, int q, double p) {
        this.name = s; this.stock = q; this.price = p; }
    ...
    public void setStock(int q) {
        this.stock = q;
        setChanged();
        notifyObservers(new Integer(q));}
    public void setPrice(int p) {
        this.price = p;
        setChanged();
        notifyObservers(new Double(p));}
}
```

Observer

- **Example – Products management (inheritance-based approach)**

Observer of
price change

```
public class PriceObserver implements Observer {  
    private double price;  
    public void update(Observable obs, Object obj){  
        if(obj instanceof Double) {  
            price = ((Double)obj).doubleValue();  
            System.out.println("PriceObserver:" + "changed to: " + price);  
        } else System.out.println("PriceObserver:" + " other changes");  
    } }  

```

Observer of
stock change

```
public class StockObserver implements Observer {  
    private integer stock;  
    public void update(Observable obs, Object obj) {  
        if(obj instanceof Integer) {  
            stock = ((Integer)obj).intValue();  
            System.out.println("PriceObserver:" + "changed to: " + stock);  
        } else System.out.println("PriceObserver:" + "other changes");  
    } }  

```

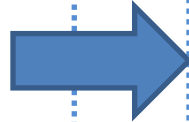
Observer

- **Example – Products management (inheritance-based approach)**

```
// create the objects
Item tvset = new Item("TV", 25, 99.99);
StockObserver so = new StockObserver();
PriceObserver po = new PriceObserver();

// add the Observers
tvset.addObserver(so);
tvset.addObserver(po);

// change Subject attributes
tvset.setPrice(89.99);
tvset.setPrice(76.45);
tvset.setStock(60);
tvset.setStock(45);
tvset.setStock(42);
```



Output

```
PriceObserver: changed to: 89.99
StockObserver: other changes
PriceObserver: changed to: 76.45
StockObserver: other changes
PriceObserver: other changes
StockObserver: changed to: 60
PriceObserver: other changes
StockObserver: changed to: 45
PriceObserver: other changes
StockObserver: changed to: 42
```

Observer

- **Implementation problems**

- How the subjects represent and keep track of their observers?
- How an observer interested in more subjects identifies the subject who sends the notification?
- How is the update triggered? Who is responsible triggering?
 - Subject, Observer, Third party component
- How much info is passed from the subject to the observers when the state changes
 - Push model – all changes (the big picture)
 - Pull model – little info (what is needed)
 - The observers subscribe for specific event

Observer

- **Problems of the inheritance-based approach**
 - No multiple inheritance in Java
 - If already extends a certain class => it cannot extend Observable
 - Solution: Use a delegation-based approach
 - Class Item or its subclasses will contain an Observable instance object
 - All Observable related behavior will be delegated to this object
 - Class Observable defines instance variable state controlled by the methods `setChanged()` and `clearChanged()`
 - These methods are protected => they cannot be queried by external objects
 - Solution: Define an Observable subclass (ObservableType)
 - Override `setChanged()` and `clearChanged()` methods as public so that they can be queried by external objects
 - Java allows this visibility change because the subclass provides more access than the superclass

Observer

- **Implementation problems**

- **java.util.Observable**

- Class that plays the role of the Subject superclass
 - The ConcreteSubjects classes should inherit from java.Util.Observable
 - Uses a Vector object to store its Observers

- **java.util.Observer**

- Interface that plays the role of Observer in the pattern structure
 - This interface must be implemented by any Observer class



From Java 9, Java.util.Observable and java.util.Observer are deprecated.

- *we study this Java support structures because they show how to implement a publish – subscriber system in which the subscribers are made aware about changes in the publisher status*

Observer

- **Example – Products management (delegation-based approach)**

```
public interface Observer {  
    void update(Item item);  
}
```

```
public class ObservableDelegate {  
    private final List<Observer> observers = new ArrayList<>();  
  
    public void addObserver(Observer observer) {  
        observers.add(observer);  
    }  
  
    public void removeObserver(Observer observer) {  
        observers.remove(observer);  
    }  
  
    public void notifyObservers(Item item) {  
        for (Observer observer : observers) {  
            observer.update(item);  
        }  
    }  
}
```

Observer

- **Example – Products management (delegation-based approach)**

```
public class Item {  
    private String name; private int stock; private double price;  
    private final ObservableDelegate delegate = new ObservableDelegate();  
    public Item(String name, int stock, double price) {  
        this.name = name; this.stock = stock; this.price = price;  
    }  
    public void addObserver(Observer observer) {  
        delegate.addObserver(observer);  
    }  
    public void removeObserver(Observer observer) {  
        delegate.removeObserver(observer);  
    }  
    public void setStock(int stock) {  
        this.stock = stock;  
        delegate.notifyObservers(this);  
    }  
    public void setPrice(double price) {  
        this.price = price;  
        delegate.notifyObservers(this);  
    }  
}
```

Observer

- Example – Products management (delegation-based approach)**

```
public class Customer implements Observer {
    private String customerName;
    public Customer(String customerName) {
        this.customerName = customerName;
    }

    @Override
    public void update(Item item) {
        System.out.println(customerName+":");
        System.out.println("Item: " +
            item.getName() +
            ", Stock: " + item.getStock() +
            ", Price: " + item.getPrice());
        System.out.println();
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Item laptop = new Item("Laptop", 10, 120);

        Customer alice = new Customer("Alice");
        Customer bob = new Customer("Bob");

        laptop.addObserver(alice);
        laptop.addObserver(bob);

        // Change stock and price
        laptop.setStock(5);
        laptop.setPrice(999.99);
    }
}
```

Observer Design Pattern



Java 11 in docs.oracle.java.com : “The event model supported by Observer and Observable is quite limited, the order of notifications delivered by Observable is unspecified, and state changes are not in one-for-one correspondence with notifications. For a richer event model, consider using the **java.beans package**. For reliable and ordered messaging among threads, consider using one of the concurrent data structures in the java.util.concurrent package. For reactive streams style programming, see the Flow API.”

- **Implementation using Java PropertyChangeListener of `java.beans package`**

```
PCLNewsAgency observable = new PCLNewsAgency();
PCLNewsChannel observer = new PCLNewsChannel();
observable.addPropertyChangeListener(observer);
observable.setNews("news");
```

Observer

```
public class PCLNewsChannel implements
PropertyChangeListener {
    private String news;
    public void propertyChange(PropertyChangeEvent evt){
        this.setNews((String) evt.getNewValue());
    }
}
```

UTCN - Programming Techniques

Usage

Observer Design Pattern

- Implementation using Java PropertyChangeListener of **java.beans package**

Observable

```
public class PCLNewsAgency {  
    private String news;  
    private PropertyChangeSupport support;  
    public PCLNewsAgency() { support = new PropertyChangeSupport(this);  
    }  
    public void addPropertyChangeListener(PropertyChangeListener pcl){  
        support.addPropertyChangeListener(pcl);  
    }  
    public void removePropertyChangeListener(PropertyChangeListener pcl){  
        support.removePropertyChangeListener(pcl);  
    }  
    public void setNews(String value) {  
        support.firePropertyChange("news", this.news, value);  
        this.news = value;  
    }  
}
```

[Source](#)

Observer

- **Java Event Model**

Mapping Java Event Model to Observer pattern participants

Event sources	GUI components	ConcreteSubject
Event listeners	Objects that subscribe to be notified of GUI events	ConcreteObserver

- EventListeners must register with EventSources to be notified when the events occur
- EventListeners should implement an interface that defines a method to be invoked by the event source when the event occurs
- Defines more listener interfaces, suitable for different types of GUI events
 - ActionListener, MouseListener, WindowListener, etc
- Each listener interface define methods that should be implemented by event listeners